

Exposé für die Projektarbeit

Entwicklung einer Custom-Engine für generative Spielwelten und Reinforcement Learning

Florian Merlau

Laurin [Nachname]

März 2026

1 Projektidee

Wir planen in unserer Projektarbeit die Entwicklung eines 2D-Top-Down-Spiels (im Stil von Minecraft), welches primär als experimentelle Testumgebung für Künstliche Intelligenz (Reinforcement Learning) konzipiert ist.

Im Gegensatz zu vielen bestehenden Projekten, die feste Level verwenden, soll unsere Architektur auf einer deterministischen, prozedural generierten "ChunkWelt basieren. Dadurch variieren die Level bei jedem Durchlauf anhand eines Seeds. Der trainierte KI-Agent kann somit keine Maps auswendig lernen, sondern muss allgemeingültige Lösungsstrategien (Ressourcen abbauen, Hindernissen ausweichen, Zielpunkte finden) entwickeln.

2 Kurzes Glossar: Wichtige Fachbegriffe

Damit die Konzepte im restlichen Exposé anschaulich und gut verständlich bleiben, hier die wichtigsten Fachbegriffe kurz und einfach erklärt:

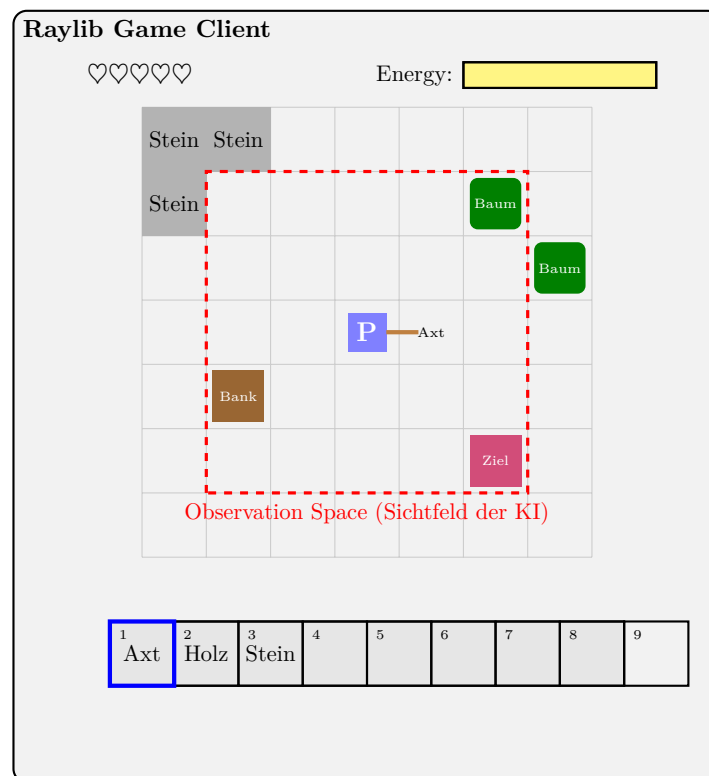
- **Reinforcement Learning (RL):** Eine Technik der Künstlichen Intelligenz, die durch pures Ausprobieren lernt – vergleichbar mit dem Training eines Hundes. Für nützliche Aktionen gibt es eine Belohnung (Reward), für sinnlose Aktionen nicht.
- **Prozedurale Generierung & Seed:** Die Spielwelt wird nicht von Menschen vorgebaut, sondern bei jedem Start von einem Algorithmus neu "gewürfelt". Der *Seed* ist dabei der zufällige Startcode: Gibt man denselben Code ein, baut der Algorithmus exakt dieselbe Welt wieder auf.
- **Observation Space:** Quasi die "Äugen" der KI. Anstatt bunte Bilder zu sehen, empfängt die KI eine flache Tabelle aus Zahlen, die beschreibt, was gerade um sie herum passiert und wie ihr Zustand ist.
- **Headless-Modus & Vektorisiert:** "Headless" (kopflös) bedeutet, dass das Spiel ohne echte Grafik völlig unsichtbar im Hintergrund simuliert wird. Dadurch ist es extrem schnell. "Vektorisiert" heißt, wir kopieren dieses unsichtbare Spiel gleich zigfach und lassen dutzende KIs gleichzeitig in Parallelwelten trainieren.

- **Sparse vs. Dense Rewards:** SSparse"(selten) bedeutet, es gibt die Belohnung erst ganz am Ende, wenn das Level geschafft ist. Das ist am Anfang oft zu schwer. "Dense"(dicht) bedeutet, es gibt schon für richtige Zwischenschritte (wie Bäume fällen) kleine Belohnungshäppchen.

3 Methodische Visualisierung: Das Spielkonzept

Um die Schnittstelle zwischen der grafischen Darstellung (für menschliche Spieler) und den Rohdaten (für die Künstliche Intelligenz) zu verdeutlichen, zeigt die folgende Skizze das Kernlayout der Engine (*StoneforgeFrontier*).

Während der visuelle Client (Raylib) die gesamte Karte sowie das UI (Leben, Energie, Inventar-Hotbar) rendert, erhält der KI-Agent als Eingabe (*Observation Space*) in erster Linie einen numerischen Array des lokalen Umfelds (hier als rote gestrichelte Box dargestellt). Um zielloses Umherirren in der prozeduralen Welt zu verhindern, erhält der Agent zusätzlich zwingend „globale“ Statusinformationen in Form eines Richtungsvektors zum generierten Ziel sowie den aktuellen Inventar- und Gesundheitsstatus.



4 Technologien und Architektur

Um maximale Performance für das CPU-intensive KI-Training zu gewährleisten, verzichten wir auf vorgefertigte Game-Engines (wie Unity) und entwickeln eine performante, maßgeschneiderte Datenarchitektur:

4.1 Engine und Schnittstellen

- **C++20 (Core Engine):** Die komplette Spielmechanik wird hochperformant in C++ geschrieben. Das ermöglicht einen "HeadlessModus, durch den die KI extrem schnell trainieren kann. Dies erlaubt es, dutzende Instanzen parallel laufen zu lassen (*Vectorized Environments*), wovon das Training massiv profitiert.
- **Python-Bindings (Pybind11 & Gymnasium):** Um die C++-Engine mit den KI-Algorithmen zu verbinden, programmieren wir Schnittstellen via *pybind11*. Das Spiel wird dadurch als standardisiertes *Gymnasium*-Environment in Python ladbar.

4.2 Prozedurale Weltengenerierung

Die Architektur der Welten benötigt eine Balance aus Varianz und Struktur. Dafür setzen wir auf einen hybriden Ansatz:

- **Layered Noise & Cellular Automata:** Die Basis für Biom-Struktur und Ressourcenverteilung bildet performantes Rauschen (Noise). Um klar definierte Hindernisse und Höhlensysteme zu schaffen, glätten wir diese Struktur mit Zellulären Automaten.
- **Flood-Fill Validierung:** Wir implementieren nach der Generierung einen Flood-Fill-Algorithmus, der garantiert, dass Zielpunkte physisch erreichbar bleiben. Ein unlösbarer Seed behindert andernfalls das KI-Training massiv.
- **Lazy Evaluation:** Die Generierung generiert Chunks strikt *on-demand* (lazy). Level-Bereiche werden erst berechnet, wenn die KI sich nähert, was den RAM-Bedarf minimiert. Um dabei zu verhindern, dass die KI später in unlösbare Sackgassen läuft, garantiert ein vorgelagerter Makro-Graph die Erreichbarkeit des Ziels auf globaler Ebene, *bevor* die feingranularen Chunks instanziiert werden.

5 KI-Algorithmen und Training

Für das Training in den prozeduralen Umgebungen nutzen wir folgendes RL-Setup:

- **PPO (Proximal Policy Optimization) [4]:** Ein robustes Policy-Gradient-Verfahren (via *Stable-Baselines3* [3]), das als Industrie-Standard gilt und stabile Lernkurven bietet.
- **Upgrades bei Stagnation:** Sollte der Agent im Grid stagnieren, planen wir funktionelle Erweiterungen wie **PPO-LSTM** (Kurzzeitgedächtnis für Navigation in teilweise beobachtbaren Umgebungen) oder **Intrinsic Curiosity / RND** [2, 1] (um den Agenten aktiv zur Exploration zu motivieren).

5.1 Reward Design und Evaluierung

Das Design der Belohnungsfunktion (Reward Function) ist maßgeblich für den Erfolg des Agenten. Um dem Problem stark verzögerter Belohnungen (*Sparse Rewards*) in weitläufigen Leveln entgegenzuwirken, setzen wir auf ein hybrides Modell: Der Agent erhält einen

signifikanten *Sparse Reward* für das Erreichen des Ziels, gestützt durch kleine *Dense Rewards* für progressionsfördernde Aktionen (z.B. den Abbau benötigter Ressourcen oder die Reduzierung der Distanz zum Ziel).

Als objektive Evaluierungsmetrik für ein „erfolgreiches“ Basis-Training definieren wir eine **Success Rate von $\geq 90\%$** über ein Set von 100 *unbekannten* Seeds. Des Weiteren evaluieren wir die Effizienz der gefundenen Pfade anhand der Minimierung der *Steps per Episode*.

6 Projektziele

6.1 Must-Have-Kriterien

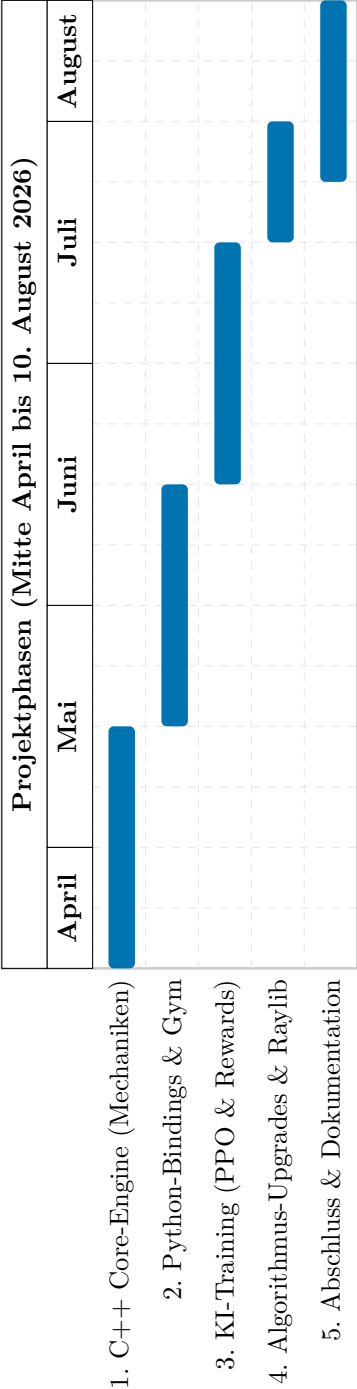
1. Entwicklung der C++-Engine mit deterministischer Chunk-Generierung.
2. Brücke zwischen C++ und Python mittels *pybind11* als vektorisiertes *Gymnasium*-Environment.
3. Basis-Training eines PPO-Agenten, der auf unbekannten Seeds sein Zielgebiet erreicht.

6.2 Nice-to-Have-Kriterien

1. Grafischer Raylib-Client zur Live-Beobachtung (wie in der Skizze zu sehen).
2. System-Upgrades wie PPO-LSTM ("Gedächtnis") oder Curiosity (RND/ICM).
3. Detailreiches Crafting-System (Holz abbauen als Voraussetzung für Werkzeuge).

7 Zeitplan

Die Phase der Umsetzung erstreckt sich über gut 4,5 Monate (Mitte April bis 10. August). Der iterative Zeitplan ist wie folgt aufgeteilt:



Literatur

- [1] Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation. In *International Conference on Learning Representations*, 2018. <https://arxiv.org/abs/1810.12894>.
- [2] Deepak Pathak, Pulkit Agrawal, Alexei A Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *International conference on machine learning*, pages 2778–2787. PMLR, 2017. <https://arxiv.org/abs/1705.05363>.
- [3] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. <http://jmlr.org/papers/v22/20-1364.html>.
- [4] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. <https://arxiv.org/abs/1707.06347>.